African Institute for Mathematical Sciences

Computer Vision

Intel Image Classification

Using Convolutional Neural Networks with PyTorch and TensorFlow

Author: Rosly MAMEKEM

mamakemrosly@gmail.com

April 2026

Contents

1 Context and Problem Statement **2**
- 1.1 Dataset . . . 2
- 1.2 Problem Statement . . . 2

2 Proposed Solution **2**
- 2.1 Convolutions and Neural Networks for Image Classification . . . 2
  - 2.1.1 Why convolutions? . . . 2
  - 2.1.2 Building blocks . . . 3
- 2.2 Model Architecture . . . 3
  - 2.2.1 PyTorch Model . . . 3
  - 2.2.2 TensorFlow/Keras Model . . . 4
  - 2.2.3 Architecture Justification . . . 4

3 Results and Performance Metrics **4**
- 3.1 Training Curves . . . 4
- 3.2 Confusion Matrices . . . 5
- 3.3 Classification Report . . . 5
- 3.4 Per-class Accuracy . . . 6
- 3.5 Summary . . . 6

4 Web Application **6**
- 4.1 Overview . . . 6
- 4.2 Technical Stack . . . 7
- 4.3 Application Interface . . . 7

5 Conclusion **8**

Context and Problem Statement

Dataset

The Intel Image Classification dataset is a benchmark dataset for natural scene recognition. It contains approximately **25,000 images** of size 150×150 pixels, distributed across **six categories**:

Buildings	**Forest**	**Glacier**	**Mountain**	**Sea**	**Street**
Natural scenes	Natural	Ice/Snow	Terrain	Water	Urban

The dataset is split into a training set (≈14,000 images) and a test set (≈3,000 images), with a roughly uniform distribution across classes.

Problem Statement

Image classification is a fundamental task in computer vision. Given a raw image as input, the objective is to assign it to one of a set of predefined categories. Formally, let $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$ be an input image with height H, width W and C colour channels. The goal is to learn a function:

$$f_\theta : \mathbb{R}^{H \times W \times C} \longrightarrow \{1, \ldots, K\}$$

that maps each image to one of $K = 6$ classes, where θ represents the learnable parameters of the model.

The main challenges are:

- **Intra-class variability:** images of the same class can look very different (e.g., a forest in summer vs. winter).
- **Inter-class similarity:** some categories share visual features (e.g., mountain and glacier both contain snow).
- **Illumination and viewpoint changes:** the appearance of a scene varies significantly with lighting conditions and camera angle.

Classical machine learning approaches (hand-crafted features such as HOG, SIFT, or LBP combined with SVM classifiers) struggle to generalise in these conditions. Deep learning, and Convolutional Neural Networks in particular, address these challenges by learning hierarchical representations directly from raw pixels.

Proposed Solution

Convolutions and Neural Networks for Image Classification

Why convolutions?

A fully connected neural network applied to a $150 \times 150 \times 3$ image would require connecting each neuron to all 67,500 input pixels. This leads to an enormous number of parameters, making the model both slow to train and prone to overfitting.

Convolutional layers solve this by exploiting two key properties of images:

- **Local connectivity:** each neuron is connected only to a small spatial region (the receptive field), defined by the kernel size.
- **Weight sharing:** the same filter is applied across all spatial positions, dramatically reducing the number of parameters.

A single convolution operation can be written as:

$$(I * K)[i, j] = \sum_m \sum_n I[i + m,\ j + n] \cdot K[m, n]$$

where I is the input feature map and K is the learned kernel.

Building blocks

A standard convolutional block used in this project consists of three operations applied in sequence:

1. **Convolution (Conv2d):** extracts local spatial features.
2. **Batch Normalisation:** normalises activations across the mini-batch, stabilising and accelerating training.
3. **ReLU activation:** introduces non-linearity via $\mathrm{ReLU}(x) = \max(0, x)$.
4. **Max Pooling:** downsamples the spatial dimensions by a factor of 2, increasing the receptive field of subsequent layers.

Model Architecture

Both models follow the same architectural philosophy: a feature extractor made of stacked convolutional blocks, followed by a fully connected classifier.

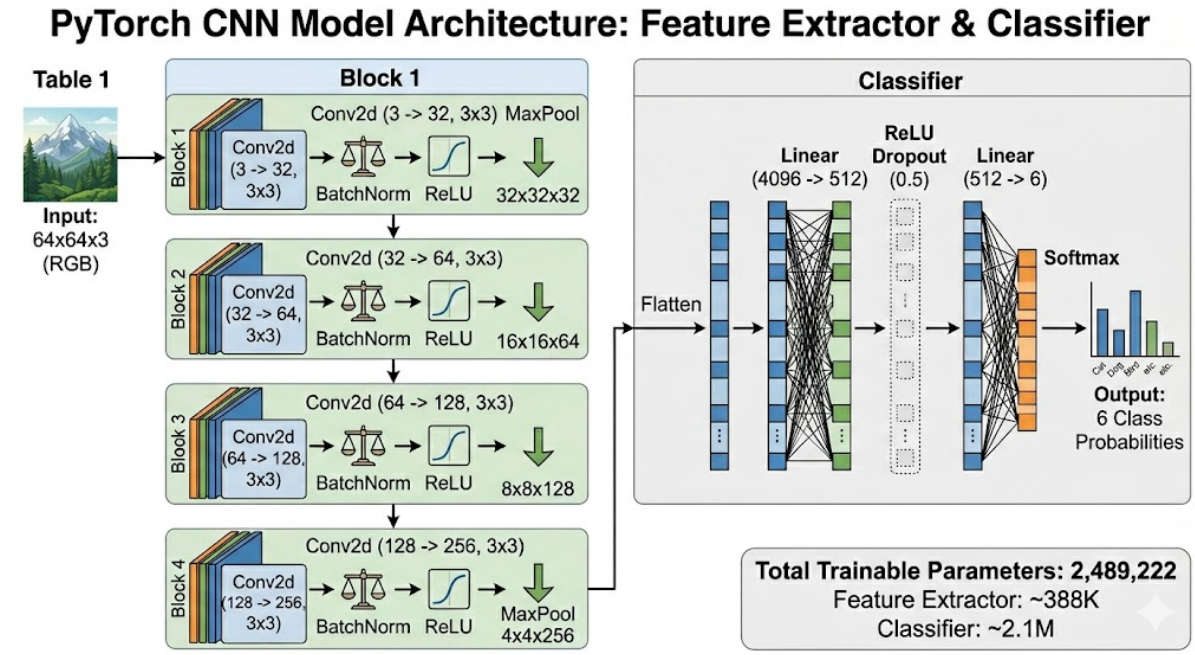
PyTorch Model

The PyTorch model is a custom CNN with four convolutional blocks and a two-layer fully connected head, trained on 64×64 images.

Table 1: *PyTorch CNN architecture*

Block	**Layer**	**Output shape**	**Params**
Block 1	Conv2d(3, 32, 3x3) + BN + ReLU + MaxPool	$32 \times 32 \times 32$	896
Block 2	Conv2d(32, 64, 3x3) + BN + ReLU + MaxPool	$16 \times 16 \times 64$	18,560
Block 3	Conv2d(64, 128, 3x3) + BN + ReLU + MaxPool	$8 \times 8 \times 128$	73,856
Block 4	Conv2d(128, 256, 3x3) + BN + ReLU + MaxPool	$4 \times 4 \times 256$	295,168
2*Classifier	Flatten + Linear(4096, 512) + ReLU + Dropout(0.5)	512	2,097,664
	Linear(512, 6) + Softmax	6	3,078
Total trainable parameters			**2,489,222**

h



TensorFlow/Keras Model

The Keras model follows a similar convolutional architecture, trained using the TensorFlow backend. Images are normalised to $[0, 1]$ before being fed into the network, and the model is compiled with the Adam optimiser and categorical cross-entropy loss.

Architecture Justification

- **Progressive depth (32, 64, 128, 256 filters):** each block doubles the number of filters, allowing the network to learn increasingly abstract features from edges and textures in early layers to semantic scene-level patterns in deeper layers.
- **Batch Normalisation:** reduces internal covariate shift and allows the use of higher learning rates without instability.
- **Dropout (0.5):** applied in the classifier to prevent overfitting by randomly deactivating 50% of neurons during training.
- **Max Pooling after each block:** halves the spatial resolution at each stage, reducing computation and increasing translational invariance.
- **Cross-entropy loss:**

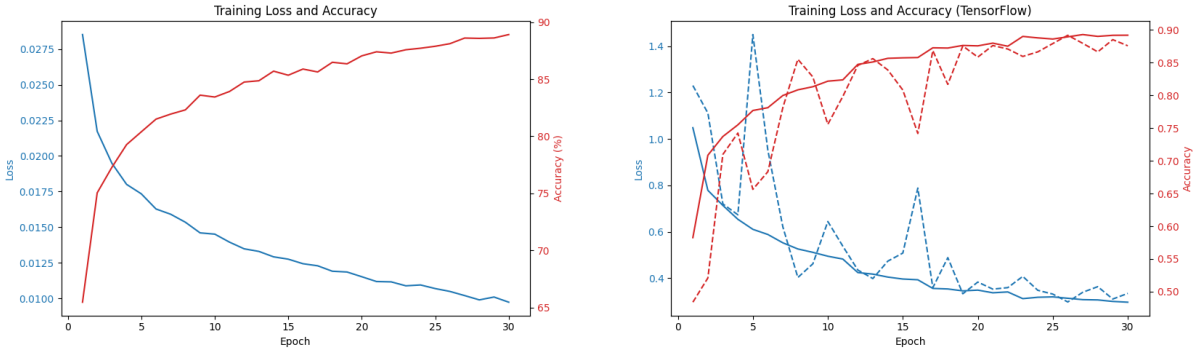
$$\mathcal{L} = -\sum_{k=1}^{K} y_k \log \hat{p}_k$$

  where y_k is the one-hot ground truth and $\hat{p}_k$ is the predicted probability for class k.

Results and Performance Metrics

Training Curves

The training curves show the evolution of loss and accuracy over epochs for both models.

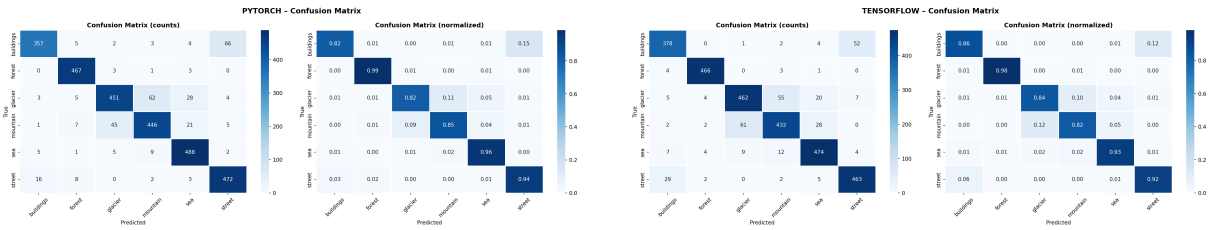


(a) *PyTorch training curves* (b) *Keras training curves*

Figure 1: *Loss and accuracy over training epochs for both models.*

Confusion Matrices

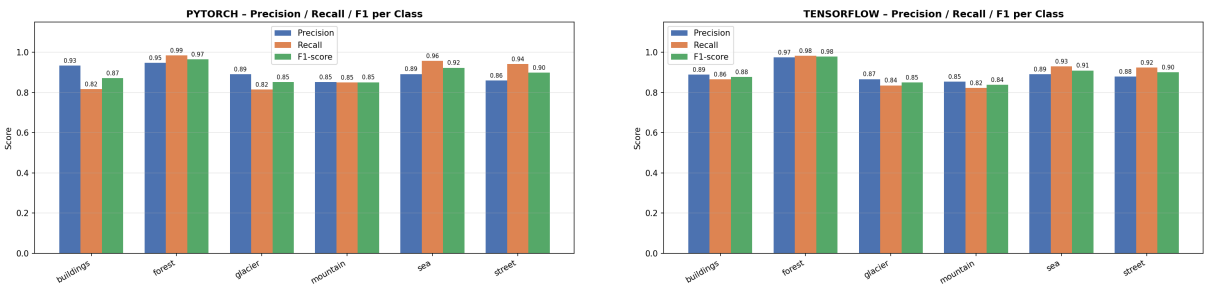
The confusion matrices show the number of correct and incorrect predictions per class on the test set.



(a) *PyTorch* (b) *TensorFlow/Keras*

Figure 2: *Confusion matrices on the test set.*

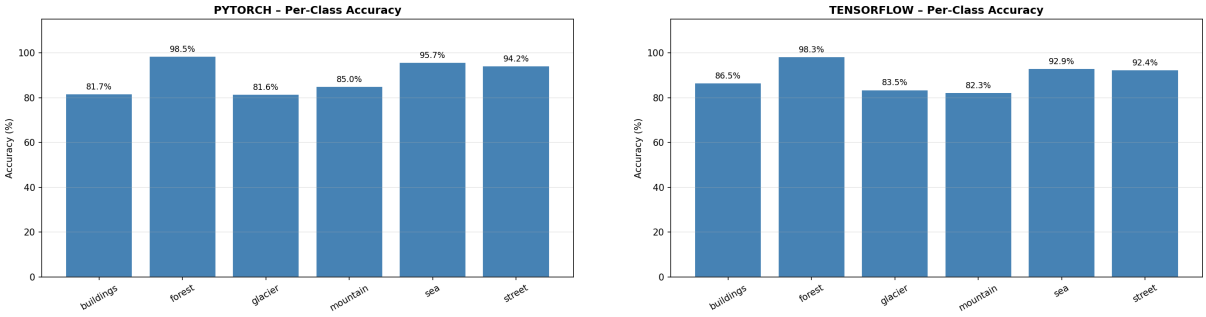
Classification Report



(a) *PyTorch* (b) *TensorFlow/Keras*

Figure 3: *Precision, recall and F1-score per class.*

Per-class Accuracy



(a) *PyTorch* **(b)** *TensorFlow/Keras*

Figure 4: *Per-class accuracy on the test set.*

Summary

Table 2: *Overall performance comparison*

Metric	PyTorch CNN	TensorFlow/Keras
Test Accuracy	91.14%	90.47%
Framework	PyTorch 2.x	TensorFlow 2.x
Input size	64×64	64×64

Both models achieve strong performance on the test set. The PyTorch model slightly outperforms the Keras model, which may be attributed to differences in regularisation strategy or the number of training epochs. Both models show good generalisation with no significant overfitting, as confirmed by the training curves.

Web Application

Overview

To make the models accessible and demonstrate their capabilities, a web application was developed using **Flask** (Python). The application serves a single-page interface where users can:

- Upload any image via drag-and-drop or file browser.
- Select the inference model (PyTorch or TensorFlow).
- Receive an instant prediction with confidence score and model metrics.
- Visualise which of the six categories was matched, with a visual highlight on the category grid.

Figure 5: *Application*

Technical Stack

Table 3: *Technical stack of the web application*

Component	Technology
Backend	Flask 3.x (Python)
ML inference	PyTorch 2.x / TensorFlow 2.x
Frontend	HTML5, Tailwind CSS, Vanilla JavaScript
Image preprocessing	Pillow, NumPy
Production server	Gunicorn

Application Interface

The interface replicates the original React design with a dark theme, a two-column layout, and interactive category cards.

Run locally

```
cd flask_app
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python app.py
# Open http://localhost:5000
```

Conclusion

This project demonstrates that a relatively compact custom CNN (around 2.5 million parameters) is sufficient to achieve over 90% accuracy on a challenging six-class natural scene classification task.

Key takeaways from this project:

- **Convolutional architectures** are well-suited for image classification because they exploit spatial locality and weight sharing.
- **Batch Normalisation and Dropout** are essential regularisation tools that improve both training stability and generalisation.
- **Both PyTorch and TensorFlow** produce competitive results with the same architectural design, confirming that the performance is driven by the architecture and training procedure rather than the framework.
- The deployment of models as a **Flask web application** provides an intuitive interface for non-technical users to interact with the models in real time.

Report prepared by Rosly MAMEKEM — AIMS Review B1, April 2026.